DESENVOLVIMENTO MOBILE · MÓDULO 20 · TRILHA KOTLIN · NÍVEL AVANÇADO

Tópicos Avançados e Projeto Final

IA no dispositivo e na nuvem, segurança, monetização, outras telas, carreira e o projeto integrador

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Integrar modelos de aprendizado de máquina no app, no aparelho e na nuvem.
- Aplicar as principais práticas de segurança mobile (OWASP MASVS).
- Conhecer os modelos de monetização e as APIs da Google Play.
- Levar o app a widgets, relógios e carros.
- Planejar, executar e apresentar o projeto final do curso.

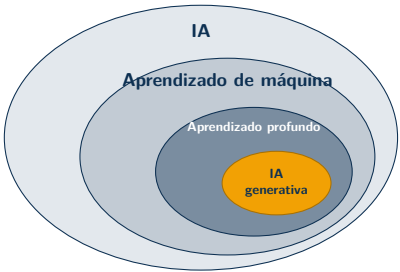
Roteiro

Neste módulo

- Fundamentos de inteligência artificial
- Fundamentos de segurança da informação
- Fundamentos de produto e negócio
- IA no app Android
- Segurança no app Android
- Monetização e outras telas
- Carreira e portfólio
- Projeto final
- Fechamento do curso

Fundamentos de inteligência artificial

IA, aprendizado de máquina e aprendizado profundo



Aprendizado de máquina (ML)

Em vez de programar as regras, o sistema **aprende** padrões a partir de exemplos (dados) e os aplica a casos novos.

Paradigma	Aprende com
Supervisionado	exemplos rotulados (leitura → "normal" ou "falha")
Não supervisionado	dados sem rótulo, buscando grupos e anomalias
Por reforço	tentativa e erro, guiado por recompensas

Aprendizado profundo

ML com redes neurais de muitas camadas; base da visão computacional, da fala e dos modelos de linguagem.

Treinamento e inferência



Treinamento

Ajustar milhões de parâmetros (**pesos**) para que o modelo erre o mínimo nos exemplos. Caro e feito uma vez (ou de tempos em tempos).

Inferência

Usar o modelo pronto para responder a uma entrada nova. É o que o app faz, milhares de vezes.

Quantização

Trocar pesos de 32 bits por inteiros de 8 bits: modelo cerca de 4 vezes menor e mais rápido, com pequena perda de precisão.

Avaliar antes de confiar

Em detecção de falhas, **precisão** (dos alertas, quantos eram reais) e **revocação** (das falhas reais, quantas foram alertadas) importam mais que a acurácia, porque falhas são raras.

Modelos de linguagem (LLMs)

LLM — *Large Language Model*

Rede neural treinada com enormes volumes de texto para prever o próximo ***token*** (pedaço de palavra). Gerando um *token* após o outro, escreve respostas, resume, traduz e segue instruções.

Termo	Significado
Janela de contexto	quanto texto o modelo considera de uma vez
Temperatura	grau de aleatoriedade da resposta
Alucinação	resposta fluente, mas falsa
RAG	buscar documentos relevantes e incluí-los na pergunta
Chamada de ferramentas	o modelo pede que o app execute uma função

Analogia

O corretor do teclado levado ao extremo: em vez da próxima palavra, continua parágrafos inteiros.

Atenção

O modelo não "sabe" se está certo: produz o texto mais provável. A resposta é uma **sugestão** que uma pessoa confere.

Dica

Com RAG, o LabMonitor responde com base nos manuais dos equipamentos, citando a fonte.

Engenharia de *prompt* e uso responsável

Estrutura de um bom *prompt*

```
Papel:     Você é assistente técnico de um
           laboratório de eletroeletrônica.
Contexto:  Leituras da bancada B3 (última hora):
           {leituras}
Tarefa:    Explique o alerta de temperatura.
Formato:   No máximo 3 frases, em português,
           e termine com uma ação recomendada.
Limites:   Se os dados não bastarem, diga isso.
```

Dica

Dê exemplos de entrada e saída esperadas (*few-shot*) e peça respostas em JSON quando o app precisar processá-las.

Injeção de *prompt*

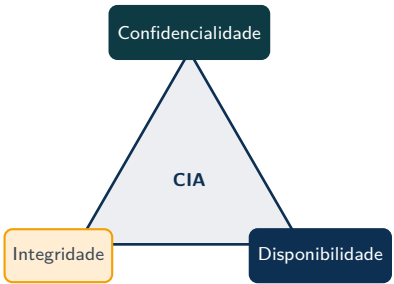
Texto vindo do usuário ou de documentos pode conter instruções ("ignore as regras anteriores..."). Nunca dê ao modelo poder de executar ações sensíveis sem confirmação e validação no servidor.

Uso responsável

- Informe que a resposta foi gerada por IA.
- Minimize dados pessoais enviados (LGPD).
- Limite custos: cada *token* é cobrado.
- Avalie com casos de teste, como qualquer código.

Fundamentos de segurança da informação

Princípios de segurança



Tríade CID (em inglês, CIA).

Propriedade	Pergunta
Confidencialidade	só quem pode vê os dados?
Integridade	os dados não foram alterados indevidamente?
Disponibilidade	o serviço funciona quando precisam dele?
Autenticidade	quem diz ser é quem é?

Princípios de projeto

Menor privilégio (cada parte com o mínimo de acesso), **defesa em profundidade** (várias camadas), **seguro por padrão** e **nunca confiar no cliente**: o app pode ser modificado por quem o instala.

Criptografia essencial

	Hash	Simétrica	Assimétrica
Ideia	resumo de tamanho fixo, impossível de reverter	a mesma chave cifra e decifra	par de chaves: pública e privada
Exemplos	SHA-256; para senhas, Argon2 ou bcrypt	AES-GCM	RSA, curvas elípticas (ECDSA, X25519)
Uso no app	integridade de arquivos, armazenar senhas no servidor	dados locais cifrados, conteúdo do canal TLS	assinatura do app, troca de chaves, *passkeys*

Como o HTTPS junta tudo

O TLS usa criptografia assimétrica para autenticar o servidor (certificado) e combinar uma chave secreta; depois cifra o tráfego com criptografia simétrica, que é muito mais rápida.

Criptografia caseira

Inventar algoritmos, reutilizar vetores de inicialização ou guardar a chave junto com o dado cifrado. Use bibliotecas e o cofre de chaves do sistema (Keystore, Keychain).

Modelagem de ameaças com STRIDE

Ameaça	Exemplo no LabMonitor	Defesa
Spoofing (falsificação)	alguém se passa por um ESP32 e envia leituras falsas	autenticar dispositivos (certificados, MQTT com senha)
Tampering (adulteração)	alterar a reserva no tráfego	TLS, validação no servidor
Repudiation (repúdio)	aluno nega ter cancelado a reserva	registros de auditoria
Information disclosure	token salvo em texto puro	armazenamento seguro
Denial of service	app em laço enviando milhares de pedidos	limites de taxa no servidor
Elevation of privilege	aluno vira administrador alterando o app	regras e papéis no servidor

Modelagem de ameaças

Analisar sistematicamente o que pode dar errado, **antes** de construir: desenhar o fluxo de dados, listar ameaças, priorizar e definir defesas.

Um pouco de história

STRIDE foi criado na Microsoft no fim dos anos 1990 e virou uma das técnicas mais ensinadas.

OWASP Mobile Top 10 e MASVS

OWASP Mobile Top 10 (2024)	
M1	uso inadequado de credenciais
M2	segurança insuficiente da cadeia de suprimentos
M3	autenticação e autorização inseguras
M4	validação insuficiente de entradas e saídas
M5	comunicação insegura
M6	controles de privacidade inadequados
M7	proteção insuficiente do binário
M8	configuração de segurança incorreta
M9	armazenamento de dados inseguro
M10	criptografia insuficiente

OWASP

Fundação aberta que publica guias de segurança. O **Top 10** lista os riscos mais comuns; o **MASVS** define os requisitos de segurança para apps, e o **MASTG** ensina a testá-los.

Grupos do MASVS

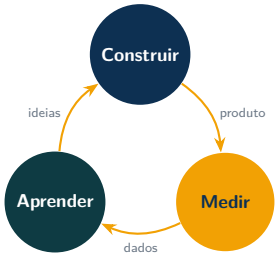
Armazenamento, criptografia, autenticação, rede, plataforma, código, resiliência e privacidade.

Dica

Use o MASVS como *checklist* de revisão antes de cada versão importante.

Fundamentos de produto e negócio

Do problema ao MVP



MVP — Produto Mínimo Viável

A menor versão do produto que permite **aprender** com usuários reais se a solução resolve o problema. Não é um produto mal feito: é um produto pequeno.

Métricas AARRR

Aquisição (chegam), **A**tivação (têm uma boa primeira experiência), **R**etenção (voltam), **R**eceita (pagam) e **R**ecomendação (indicam).

Um pouco de história

O ciclo construir–medir–aprender foi popularizado por Eric Ries no livro *A Startup Enxuta* (2011).

Modelos de receita e métricas do negócio

Modelo	Como gera receita
Pago	paga-se para instalar
Freemium	grátis, com recursos pagos
Assinatura	cobrança recorrente
Anúncios	exibições e cliques
B2B / institucional	a organização paga pelo serviço
Público / acadêmico	financiado por projeto ou órgão

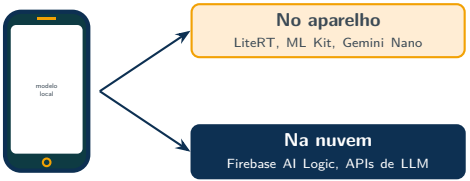
Métrica	Definição
ARPU	receita média por usuário ativo no período
Churn	% de assinantes que cancelam por mês
LTV	receita esperada de um usuário durante toda a relação; aproximação: ARPU mensal $\times$ margem $\div$ *churn* mensal
CAC	custo para adquirir um usuário (marketing $\div$ novos usuários)

Conta rápida

ARPU de R$ 10/mês, margem de 70 % e *churn* de 5 %/mês: LTV $\approx 10 \times 0{,}7 \div 0{,}05 =$ R$ 140. Gastar R$ 200 para adquirir cada usuário dá prejuízo.

IA no app Android

Onde roda o modelo?



	Aparelho	Nuvem
Latência	muito baixa	depende da rede
Offline	sim	não
Privacidade	dados ficam no aparelho	dados saem
Custo	nenhum por uso	por requisição
Tamanho do modelo	pequeno (KB–MB)	grande

ML Kit

Modelos prontos do Google: texto em imagem (OCR), códigos de barras, rostos, rótulos, tradução.

LiteRT

Novo nome do **TensorFlow Lite**: roda o **seu** modelo (`.tflite`) com aceleração por GPU/NPU.

Gemini Nano

LLM do sistema em aparelhos compatíveis, acessível pelas APIs de IA generativa do ML Kit.

Detectando anomalias nas leituras

1) Linha de base estatística (z-score)

```
fun Flow<Double>.anomalias(
  janela: Int = 60, limiar: Double = 3.0
): Flow<Double> = flow {
  val buf = ArrayDeque<Double>()
  collect { x ->
    if (buf.size == janela) {
      val m = buf.average()
      val dp = sqrt(buf.sumOf { (it - m).pow(2) } /
      janela)
      if (dp > 0 && abs(x - m) / dp > limiar) emit(x)
      buf.removeFirst()
    }
    buf.addLast(x)
  }
}
```

2) Modelo próprio com LiteRT

```
class DetectorAnomalia(ctx: Context) {
  private val modelo = Interpreter(
    FileUtil.loadMappedFile(ctx, "anomalia.tflite"))

  // janela de 60 leituras normalizadas -> prob.
  fun probabilidade(janela: FloatArray): Float {
    val entrada = arrayOf(janela)          // [1, 60]
    val saida = Array(1) { FloatArray(1) } // [1, 1]
    modelo.run(entrada, saida)
    return saida[0][0]
  }
}
```

Treino em Python (Keras) e conversão

```
conv = tf.lite.TFLiteConverter.from_keras_model(m)
conv.optimizations = [tf.lite.Optimize.DEFAULT]
open("anomalia.tflite", "wb").write(conv.convert())
```

Dica

Sempre compare o modelo de IA com uma linha de base simples como esta. Às vezes ela basta.

IA na nuvem e boas práticas

Firebase AI Logic

SDK para chamar modelos Gemini a partir do app **sem expor chaves**: o Firebase intermedeia a chamada e o **App Check** garante que só o seu app usa a cota.

Ideias para o LabMonitor

- "Explique este alerta" a partir das últimas leituras.
- Fotografar a placa do equipamento e preencher o cadastro (OCR).
- Resumo semanal de uso das bancadas em linguagem natural.

Cuidados

- LLMs erram: mostre como sugestão e peça confirmação.
- Dados pessoais só com necessidade e aviso (LGPD).
- Limite o uso e trate falhas da rede.
- Quantize modelos locais (*int8*).

Dica

Teste o modelo no aparelho mais fraco do seu público.

Segurança no app Android

Revisão MASVS de um app Android

Área	Práticas no Android
Armazenamento	dados sensíveis só no armazenamento interno; chaves no **Android Keystore**; nada sensível em logs ou backups
Criptografia	algoritmos padrão (AES-GCM, RSA/EC) via Keystore; nunca criptografia "caseira"
Autenticação	tokens de curta duração; biometria com `BiometricPrompt`; *passkeys* com Credential Manager
Rede	HTTPS sempre; *network security config*; *pinning* só com plano de rotação
Plataforma	expor o mínimo de componentes; validar *intents* e *deep links*; WebView sem JavaScript desnecessário
Código	dependências atualizadas; R8; nenhum segredo no APK
Resiliência	**Play Integrity API** para verificar app e aparelho no servidor

O cliente nunca é confiável: toda regra importante precisa ser verificada também no servidor.

Monetização e outras telas

Modelos de monetização

Modelo	Como funciona
Pago	usuário paga para baixar
Freemium	grátis com recursos pagos (compras no app)
Assinatura	cobrança recorrente por acesso
Anúncios	receita por exibição (AdMob)
B2B / institucional	a organização paga pelo app e pelo serviço
Gratuito	projetos acadêmicos, públicos e de extensão

Google Play Billing

Bens digitais vendidos dentro de apps distribuídos pela Play usam o sistema de faturamento da Play, que retém uma taxa de serviço. Bens físicos e serviços prestados fora do app seguem outras regras.

Atenção

Valide as compras **no servidor** com a API da Play antes de liberar o conteúdo.

Dica

Para apps institucionais, o modelo comum é gratuito, sem anúncios, com o custo coberto pelo projeto.

Além do celular



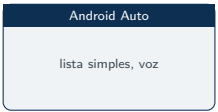
Widgets (Glance)

Informação na tela inicial, escrita com uma API no estilo Compose.



Wear OS

Apps e *tiles* no relógio com Compose for Wear OS; ideal para alertas rápidos.



Carro e TV

Android Auto, Automotive e TV usam *templates* e regras de segurança próprias.

Dica

A arquitetura em camadas permite reaproveitar repositórios e casos de uso em todas essas superfícies; só a camada de UI muda.

Carreira e portfólio

Do curso ao mercado

Portfólio que convence

- 2–3 apps completos no GitHub, com README, capturas e arquitetura explicada.
- Pelo menos um app publicado na Play (mesmo em teste aberto).
- Testes e CI visíveis no repositório.
- Um projeto com hardware (BLE/IoT) é um diferencial raro.

Continue aprendendo

- Código-fonte do *Now in Android* e dos *samples* oficiais.
- Blog *Android Developers* e as notas de versão do Jetpack.
- Comunidades: GDG, Kotlin User Groups, eventos locais.
- Contribua com bibliotecas de código aberto.

Assistentes de IA

Use-os para acelerar, mas entenda e revise cada linha: você responde pelo código que publica.

Projeto final

Projeto final: requisitos

Requisitos mínimos

1. 4+ telas com Navigation 3 e Material 3.
2. MVVM com repositórios e Hilt.
3. Room e DataStore.
4. API REST ou Firebase.
5. Offline nas telas principais.
6. Um recurso do dispositivo (BLE, câmera...).
7. Testes de ViewModels e casos de uso.
8. Acessibilidade e tema escuro.

Sugestões de tema

- Concluir e publicar o **LabMonitor**.
- Controle de empréstimo de equipamentos do laboratório via QR code.
- Medidor de consumo de energia com ESP32 e sensor de corrente.
- App de acompanhamento de estágio ou TCC.

Dica

Resolva um problema real de alguém que você conhece.

Entregáveis, cronograma e avaliação

Semana	Marco
1	proposta: problema, público, telas (protótipo) e arquitetura
2	navegação, UI e dados locais
3	rede/Firebase, recurso do dispositivo e offline
4	testes, acessibilidade, release e apresentação

Entregáveis

Repositório GitHub com README, APK/AAB de release, vídeo de 3 minutos e apresentação de 10 minutos.

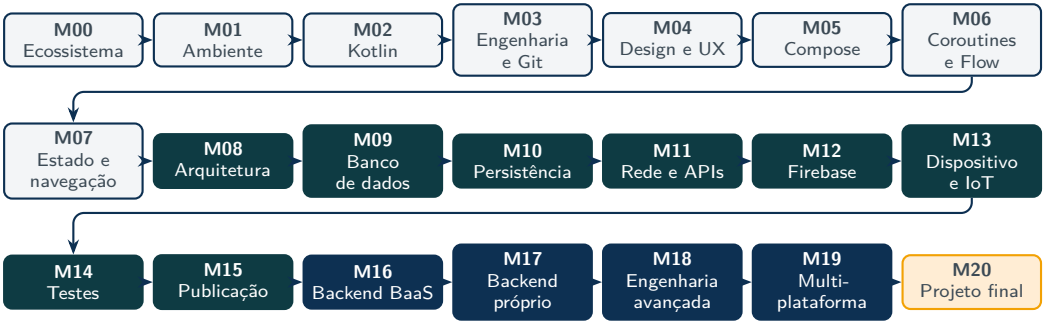
Critério de avaliação	Peso
Funcionalidades e requisitos atendidos	25 %
Arquitetura e qualidade do código	20 %
Experiência do usuário, UI e acessibilidade	15 %
Testes automatizados e CI	15 %
Uso de recursos do dispositivo / integração	10 %
Documentação (README, diagramas)	5 %
Apresentação e demonstração	10 %
Total	**100 %**

Dica

Pesos sugeridos: ajuste conforme o plano de ensino da disciplina.

Fechamento do curso

A jornada completa



O que você construiu

Um app Android completo e publicado, com arquitetura profissional, dados locais e remotos, *backend* próprio e terceirizado, integração com hardware, testes e entrega contínua, além da base para levá-lo a outras plataformas.

Glossário

Termo	Significado
Aprendizado de máquina	técnica em que o sistema aprende padrões a partir de dados
Inferência	uso de um modelo treinado para responder a entradas novas
Quantização	redução da precisão dos pesos (ex.: 32 para 8 bits) para encolher o modelo
LLM	modelo de linguagem que gera texto prevendo o próximo *token*
RAG	técnica que inclui documentos recuperados na pergunta ao LLM
STRIDE	classificação de ameaças: falsificação, adulteração, repúdio, vazamento, negação de serviço, elevação de privilégio
MASVS	padrão da OWASP com os requisitos de segurança de apps móveis
MVP	menor versão do produto que permite aprender com usuários reais
LTV / CAC	valor de um cliente ao longo da relação / custo para adquiri-lo

Questões de revisão

1. Diferencie treinamento de inferência. Onde cada um costuma acontecer num app?
2. Por que precisão e revocação são mais úteis que a acurácia para avaliar um detector de falhas?
3. O que é alucinação em LLMs e como o app deve lidar com ela?
4. Por que a chave de API de um LLM não pode ficar dentro do app? Qual a alternativa?
5. Aplique STRIDE a uma funcionalidade do seu projeto final e proponha uma defesa por ameaça.
6. Por que a compra no app deve ser validada no servidor?
7. Qual o risco do *certificate pinning* e como mitigá-lo?
8. Calcule o LTV com ARPU de R$ 8/mês, margem de 60 % e *churn* de 4 %/mês.

Exercícios finais

Exercício 1 — IA

Gere 24 h de leituras simuladas com picos artificiais e compare o detector por z-score com um modelo treinado em Python e convertido para LiteRT: quantos picos cada um encontra?

Exercício 2 — Segurança

Faça uma revisão do LabMonitor com o checklist MASVS do slide e registre ao menos três correções em *issues* no GitHub.

Exercício 3 — Widget

Crie um widget Glance que mostre a última temperatura da bancada favorita e abra o app ao ser tocado.

Projeto final

Escolha o tema, entregue a proposta na semana 1 e siga o cronograma de 4 semanas.

Para aprofundar

- IA no Android: `https://developer.android.com/ai`
- LiteRT: `https://ai.google.dev/edge/litert` ML Kit: `https://developers.google.com/ml-kit`
- Firebase AI Logic: `https://firebase.google.com/docs/ai-logic`
- OWASP MASVS: `https://mas.owasp.org/MASVS/`
- Segurança no Android: `https://developer.android.com/privacy-and-security/security-tips`
- Google Play Billing: `https://developer.android.com/google/play/billing`
- Glance (widgets): `https://developer.android.com/develop/ui/compose/glance`

Respostas das questões de revisão

1. Treinamento ajusta os pesos com muitos dados (servidor com GPU); inferência usa o modelo pronto para uma entrada nova (no aparelho ou na nuvem, a cada uso).
2. Falhas são raras: um detector que nunca alerta acerta quase sempre, mas é inútil. Precisão mede alarmes falsos; revocação mede falhas não detectadas.
3. É uma resposta fluente e falsa. O app deve apresentá-la como sugestão, citar fontes quando houver (RAG) e pedir confirmação humana para decisões.
4. O binário pode ser extraído e a chave, usada por terceiros às suas custas. A chamada deve passar por um backend ou serviço intermediário (Firebase AI Logic com App Check).
5. Exemplo para a reserva: falsificação de usuário (autenticação), adulteração da reserva (TLS e validação no servidor), elevação de privilégio (papéis verificados no servidor).
6. O app pode ser modificado para fingir a compra; só o servidor, consultando a loja, confirma que o pagamento existe.
7. Se o certificado mudar, o app para de se conectar. Mitigação: fixar a chave da autoridade intermediária, manter um certificado reserva e planejar a troca.
8. LTV $\approx 8 \times 0{,}6 \div 0{,}04 = \text{R\$}\,120$.

Obrigado!

Prof. Dr. Francisco dos Santos Viana

Fim do curso Desenvolvimento Mobile — trilha Kotlin